

Luxury Car Rental Database

How I did the research

We looked at Enterprise's rules for renting exotic cars. They check your driver's license, verify insurance, and take a security deposit. We're using their process as a model. But we're not copying every rule they have, we simplified some things.

Source: [Enterprise — Requirements for Renting an Exotic Car](#), accessed September 26, 2026.

What the system covers

This system handles one rental location. Each booking is for one car and one main renter. It tracks customers, cars, bookings, actual rentals, and payments. It does *not* handle employees, multiple locations, insurance claims, or detailed maintenance schedules.

Customers can pay before they pick up the car, and payments are tied to bookings. Deposit amounts get recorded separately from payments. We're not handling the actual credit card charges or releasing deposits that's outside this project.

What data I need to store

1. **Customers** — ID, name, contact info, driver's license number, where the license was issued, and when it expires.
2. **Vehicles** — ID, VIN, make, model, year, daily rate, and whether it's in service.
3. **Bookings** — who's renting, which car, pickup and return date/time, the daily rate they agreed to, deposit amount, and booking status.
4. **Rental agreements** — which booking it's tied to, actual pickup and return times, and odometer readings at both ends. Return info can stay blank while the rental's still going.
5. **Verification** — whether we checked the license and insurance for that rental.
6. **Payments** — payment ID, which booking it's for, date/time, amount, and how they paid. A booking can have more than one payment.

Rules the system has to follow

- You should be able to search for cars that are free during a certain time window.
- Two active bookings can't overlap on the same car. Canceled bookings don't count, so they don't block the car.
- A return date has to come after the pickup date. Same for the actual pickup and return times.
- We keep completed and canceled bookings on file, so staff can look up a customer's or car's history.
- If a car's daily rate changes later, we still keep the rate that was agreed on for each specific booking.
- The odometer reading at return has to be equal to or higher than the reading at pickup.

Searches the database needs to handle

- Find available cars for a given time period.
- Pull up a customer's bookings and rental history.
- See which cars are out right now.
- Show all payments made on a booking.
- Flag rentals that are still out past their scheduled return time.

How things connect

- One customer can make many bookings. Each booking belongs to one customer.
- One car can show up in many bookings over time. Each booking is for one car.
- A booking can turn into one rental agreement, or none. Each rental agreement belongs to one booking.
- A booking can have many payments, or none. Each payment belongs to one booking.

still need to figure out

Before we build this, we need to nail down: the cancellation and refund policy, how we calculate rental days, and how much turnaround time we need between bookings. Until we get more research, just write these down as decisions the project team needs to make.

Luxury Car Rental — Business Rules

Customers and bookings

- BR1. Every customer gets a unique ID.
- BR2. A customer can have zero bookings or a bunch of them. But each booking only has one customer attached — the main renter.

Vehicles and bookings

- BR3. Every car has a unique ID and a unique VIN.
- BR4. Each booking is for exactly one car. A car can show up in zero bookings or a bunch of them over time.
- BR5. A car can't have two confirmed or ongoing bookings that overlap. Canceled bookings don't count against it.
- BR6. If a car's marked out of service, you can't confirm a new booking for it.

Dates and pricing

- BR7. Every booking gets a unique ID and records the scheduled pickup and return date/time. Return has to be after pickup.
- BR8. We lock in the daily rate and deposit amount at the time of booking. If the car's rate changes later, it doesn't touch bookings that are already locked in.
- BR9. Rental time is billed in full 24-hour blocks. Even a few extra minutes counts as a whole extra day. Minimum charge is one day. This is just how we're simplifying it for the project.
- BR10. A booking runs from pickup time up to (but not including) return time. That means one booking can start the exact moment another one ends — no gap needed. We're not modeling cleaning or turnaround time here.

Rental agreements

- BR11. A booking can turn into one rental agreement, or none at all. Each rental agreement is tied to exactly one booking.
- BR12. The rental agreement gets created when the customer actually picks up the car. A canceled booking can never turn into a rental agreement.
- BR13. License and insurance checks have to be done *before* the car goes out.
- BR14. The rental agreement logs the actual pickup time and the odometer reading at pickup. Return time and return odometer stay blank until the car comes back.
- BR15. The car can't come back before it left, and the return odometer can't read lower than the pickup odometer.

Payments and keeping records

- BR16. Every payment has a unique ID and belongs to one booking. A booking can have zero payments or several.
- BR17. Every payment needs a positive amount, a date/time, and a payment method. Customers can pay before they even pick up the car.
- BR18. Rental payments and security deposits are two different things. We record how much deposit is required, but we don't actually process holding or releasing that deposit — that's not part of this project.
- BR19. We keep canceled and completed bookings in the system for record-keeping. We're not handling refunds or cancellation fees here.

Database design

The Luxury Car Rental database. It has five parts: Customer, Vehicle, Reservation, Rental Agreement, and Payment.

We keep customer and vehicle details separate from the booking and rental info. Same for payments. This way, we're not storing the same data twice.

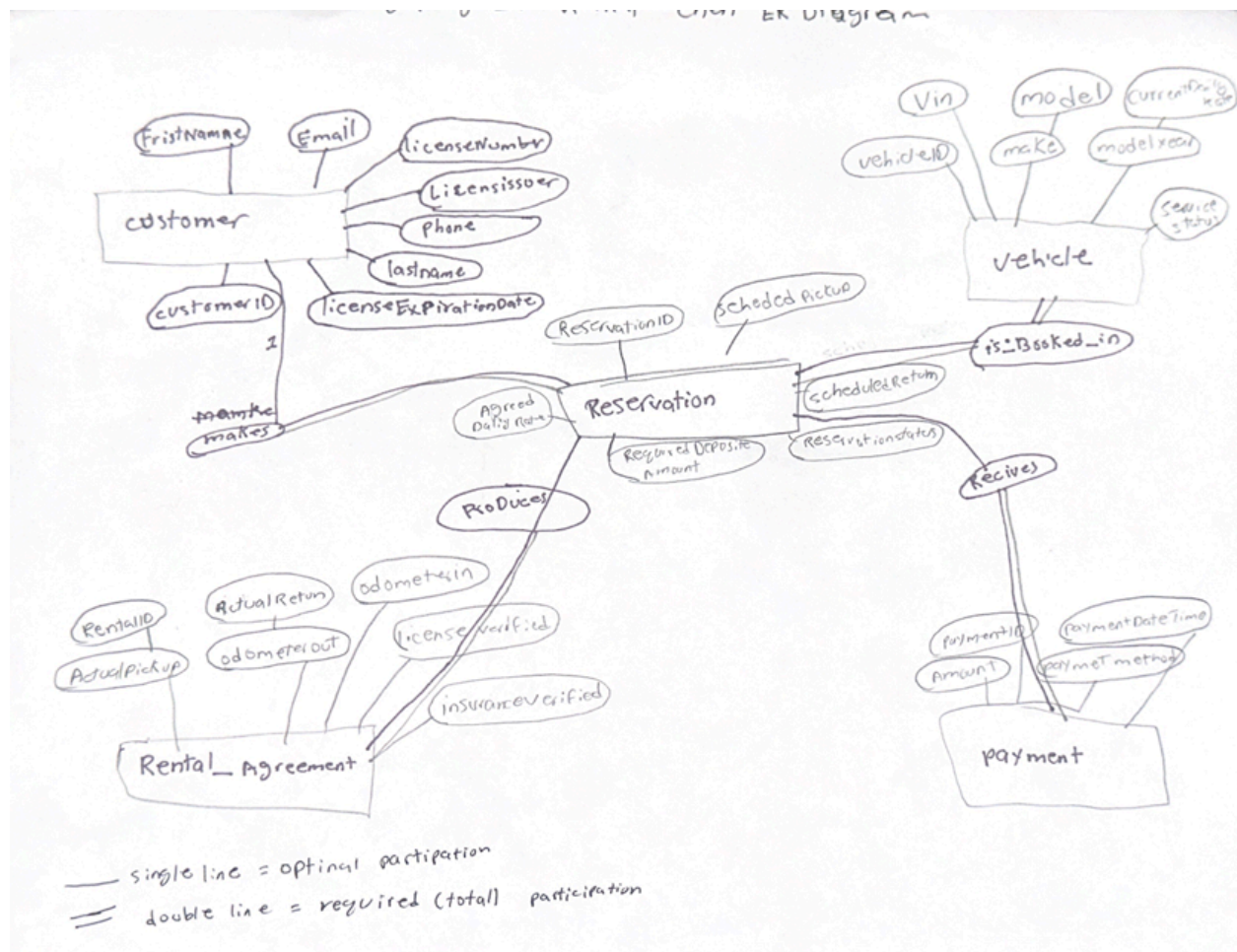


Figure 1

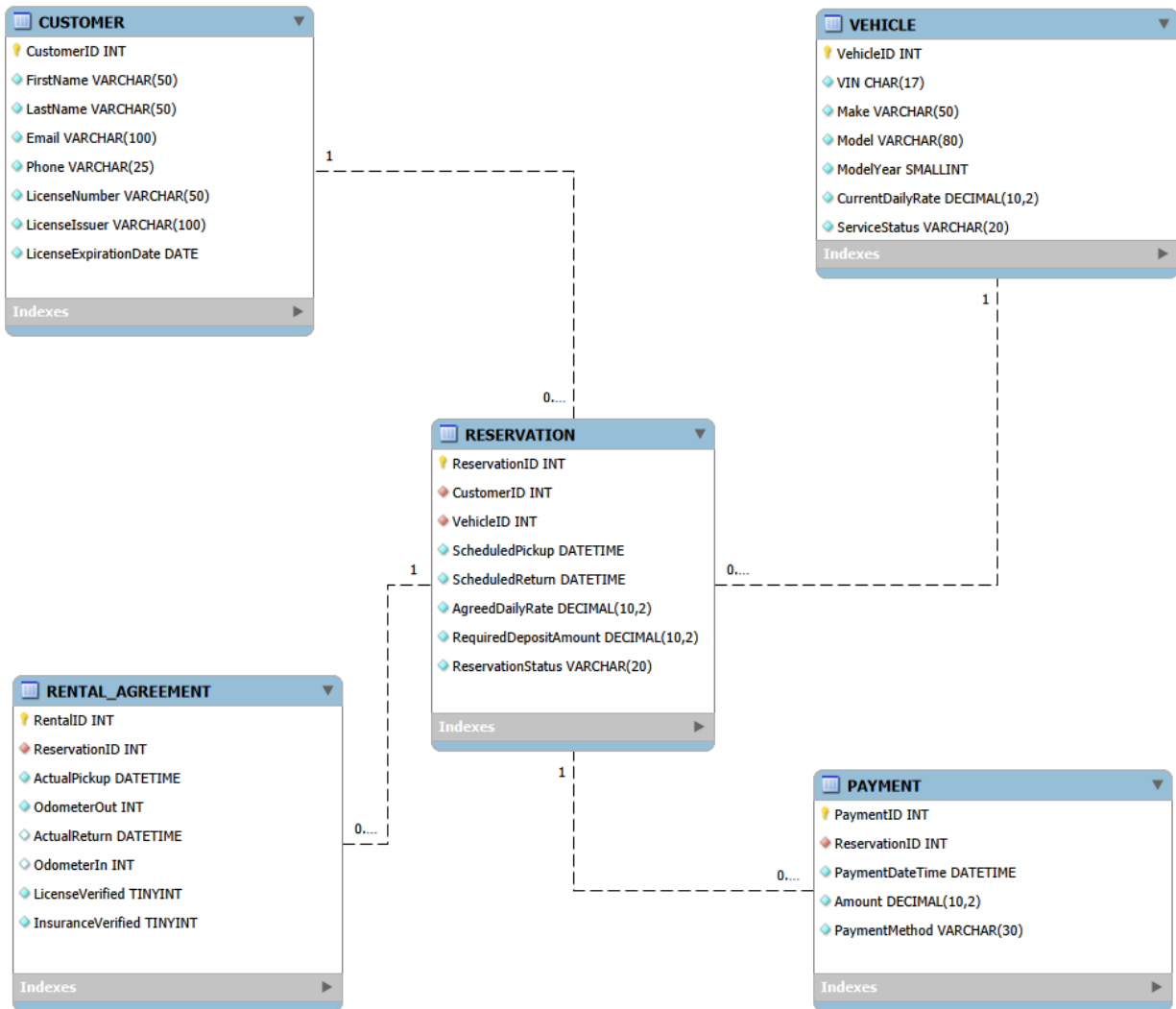


Figure 2

Implementation

The Luxury Car Rental database was designed using MySQL Workbench and implemented on MySQL Server 8.0.46. The database schema, `luxury_car_rental`, contains five tables.

Each table uses a primary key to identify its records. Foreign keys maintain relationships between tables. Unique constraints prevent duplicate vehicle VINs and ensure that each reservation has at most one rental agreement.

The screenshot shows the MySQL Workbench interface with the 'luxury_car_rental' database selected. The SQL Editor displays the following script:

```

1 USE luxury_car_rental;
2
3 ALTER TABLE VEHICLE
4 ADD CONSTRAINT chk_vehicle_rate
5 CHECK (CurrentDailyRate > 0);

```

The 'Result Grid' shows the constraints for the 'luxury_car_rental' database:

TABLE_NAME	CONSTRAINT_NAME	ENFORCED
payment	chk_payment_amount	YES
rental_agreement	chk_odometer_in	YES
rental_agreement	chk_odometer_out	YES
rental_agreement	chk_rental_dates	YES
rental_agreement	chk_rental_verified	YES
reservation	chk_reservation_dates	YES
reservation	chk_reservation_deposit	YES
reservation	chk_reservation_rate	YES
reservation	chk_reservation_status	YES
vehicle	chk_vehicle_rate	YES
vehicle	chk_vehicle_status	YES

The 'Output' window shows the execution results of the SQL script:

```

# Time Action Message
19 20:46:42 SELECT VERSION() LIMIT 0, 1000 1 row(s) returned
20 20:47:15 SHOW TABLES FROM luxury_car_rental 5 row(s) returned
21 20:47:15 SELECT VERSION() LIMIT 0, 1000 1 row(s) returned
22 20:47:15 SELECT @@SESSION.unique_checks AS UniqueChecks, @@SESSION.foreign_key_checks AS ForeignKeyChecks LIMIT 0, 1000 1 row(s) returned
23 20:49:20 USE luxury_car_rental 0 row(s) affected
24 20:49:20 ALTER TABLE VEHICLE ADD CONSTRAINT chk_vehicle_rate CHECK (CurrentDailyRate > 0); ADD CONSTRAINT chk_vehicle_status CHE... 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
25 20:49:20 ALTER TABLE RESERVATION ADD CONSTRAINT chk_reservation_dates CHECK (ScheduledReturn > ScheduledPickup); ADD CONSTRAIN... 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
26 20:49:20 ALTER TABLE RENTAL_AGREEMENT ADD CONSTRAINT chk_rental_dates CHECK (ActualReturn IS NULL OR ActualReturn >= Actual... 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
27 20:49:20 ALTER TABLE PAYMENT ADD CONSTRAINT chk_payment_amount CHECK (Amount > 0) 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
28 20:49:20 SELECT TABLE_NAME, CONSTRAINT_NAME, ENFORCED FROM information_schema.TABLE_CONSTRAINTS WHERE CONSTRAINT_SCHEMA... 11 row(s) returned

```

A query of `information_schema.TABLE_CONSTRAINTS` confirmed that all 11 CHECK constraints were marked `ENFORCED = YES`. Tests also confirmed that the database rejected a negative payment, a reservation with reversed dates, and a second rental agreement for the same reservation.

Testing

Limit to 1000 rows

```
63 (4, 3, '2026-09-26 15:00:00', 200.00, 'CreditCard');
64
65 • SELECT 'CUSTOMER' AS TableName, COUNT(*) AS RowCount FROM CUSTOMER
66 UNION ALL
67 SELECT 'VEHICLE', COUNT(*) FROM VEHICLE
68 UNION ALL
69 SELECT 'RESERVATION', COUNT(*) FROM RESERVATION
70 UNION ALL
71 SELECT 'RENTAL_AGREEMENT', COUNT(*) FROM RENTAL_AGREEMENT
72 UNION ALL
73 SELECT 'PAYMENT', COUNT(*) FROM PAYMENT;
```

Result Grid

TableName	RowCount
CUSTOMER	3
VEHICLE	4
RESERVATION	4
RENTAL_AGREEMENT	2
PAYMENT	4

Limit to 1000 rows

```
44 FROM VEHICLE v
45 WHERE v.ServiceStatus = 'Operational'
46 AND NOT EXISTS (
47     SELECT 1
48     FROM RESERVATION r
49     WHERE r.VehicleID = v.VehicleID
50     AND r.ReservationStatus IN ('Confirmed', 'Ongoing')
51     AND r.ScheduledPickup < @RequestedReturn
52     AND r.ScheduledReturn > @RequestedPickup
53 )
54 ORDER BY v.VehicleID;
```

Result Grid

VehicleID	Make	Model	CurrentDailyRate
1	Porsche	911	450.00
2	Mercedes-Benz	S-Class	350.00
NULL	NULL	NULL	NULL

The screenshot shows a SQL script in a text editor with the following content:

```

2
3 • START TRANSACTION;
4
5 • INSERT INTO PAYMENT
6 (ReservationID, PaymentDateTime, Amount, PaymentMethod)
7 VALUES
8 (1, '2026-09-26 16:00:00', -50.00, 'CreditCard');
9 • ROLLBACK;
10 • START TRANSACTION;
11
12 • INSERT INTO RESERVATION
13 (CustomerID, VehicleID, ScheduledPickup, ScheduledReturn,
14 AgreedDailyRate, RequiredDepositAmount, ReservationStatus)
15 VALUES
16 (1, 1, '2026-11-10 10:00:00', '2026-11-09 10:00:00',
17 450.00, 1000.00, 'Confirmed');
18 • ROLLBACK;
19 • START TRANSACTION;
20
21 • INSERT INTO RENTAL_AGREEMENT
22 (ReservationID, ActualPickup, OdometerOut,
23 LicenseVerified, InsuranceVerified)
24 VALUES
25 (1, '2026-09-20 10:00:00', 12000, 1, 1);
26 • ROLLBACK;

```

The output window shows the following results:

#	Time	Action	Message	Duration / Fetch
13	23:05:24	USE luxury_car_rental		0.000 sec
14	23:05:24	START TRANSACTION	0 row(s) affected	0.000 sec
15	23:05:24	INSERT INTO PAYMENT (ReservationID, PaymentDateTime, Amount, PaymentMethod) VALUES (1, '2026-09-26 16:00:00', -50.00, 'CreditCard')	Error Code: 3819 Check constraint 'chk_payment_amount' is violated.	0.016 sec
16	23:06:27	ROLLBACK	0 row(s) affected	0.000 sec
17	23:07:16	START TRANSACTION	0 row(s) affected	0.000 sec
18	23:07:16	INSERT INTO RESERVATION (CustomerID, VehicleID, ScheduledPickup, ScheduledReturn, AgreedDailyRate, RequiredDepositAmount, ReservationStatus)	Error Code: 3819 Check constraint 'chk_reservation_dates' is violated.	0.015 sec
19	23:07:20	ROLLBACK	0 row(s) affected	0.000 sec
20	23:07:46	START TRANSACTION	0 row(s) affected	0.000 sec
21	23:07:46	INSERT INTO RENTAL_AGREEMENT (ReservationID, ActualPickup, OdometerOut, LicenseVerified, InsuranceVerified) VALUES (1, '2026-09-20 10:00:00', 12000, 1, 1)	Error Code: 1062 Duplicate entry '1' for key 'rental_agreement.ReservationID_UNIQUE'	0.016 sec
22	23:08:04	ROLLBACK	0 row(s) affected	0.000 sec

Limitations and Improvements

The availability query checks for date conflicts by comparing a new request against existing reservations. But right now, nothing stops a double booking from actually going through. A future version could add a booking procedure that uses transactions: check availability first, then accept the reservation, all as one step.

Two other things aren't checked yet either. A vehicle marked `OutOfService` shouldn't be rentable. And a canceled reservation shouldn't turn into a rental agreement. Both would need extra rules, either in the database or the application.

Refunds aren't part of this project. The `PAYMENT` table tracks money coming in, not money going back out. A later version could add that records and rules for refunds and deposit returns.

Conclusion

The Luxury Car Rental database handles customer records, vehicle records, reservations, rental agreements, and payments. Each reservation links to one customer and one vehicle. The database tracks what actually happens during a rental, and it lets a reservation have more than one payment.

It's built in MySQL Workbench, running on MySQL Server 8.0.46. Primary keys, foreign keys, unique constraints, and 11 CHECK constraints keep the data clean. There are sample queries too reservation details, payment totals, active rentals, and vehicle availability by date. I tested it, and it correctly rejected bad payment amounts, reservation dates in the wrong order, and duplicate rental agreements.

This covers the basics of running a luxury car rental business. Next steps: stop double-bookings automatically, check that vehicle and reservation statuses stay consistent, and add support for refunds.

References

1. **CIS 344 Project 1 Assignment.** *Project_1.pdf*. Course requirements and submission guidelines.
2. **CIS 344 Chapter 3 Lecture Slides.** *Chapter03 (1).ppt*. Entity–relationship modeling, attributes, keys, relationships, and participation constraints.
3. **CIS 344 Chapter 4 Lecture Slides.** *Chapter04 (1).ppt*. Enhanced entity–relationship modeling concepts.
4. **Enterprise Rent-A-Car.** “[Requirements for Renting an Exotic Car](#).” Background research on license, insurance, payment, and deposit requirements. These requirements informed the project’s assumptions and are specific to Enterprise.